

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет по Домашней работе №1
“CSS Flexbox, CSS Grid”

Выполнила:

Бахарева М.А.

Группа 3342

Проверил:

Добряков Д. И.

Санкт-Петербург

2024 г.

Задача

Пройти три игры:

- <https://flexboxfroggy.com/#ru>
- <https://cssgridgarden.com/#ru>
- https://learngitbranching.js.org/?locale=ru_RU (основы работы с git, достаточно выполнить первые 4 урока)

Собрать отчёт по шаблону (<https://docs.google.com/document/d/1zgXRYAh9iO21NDcp5Cua3mOiNRWbAVZJBfgGiqGsMsU/edit?usp=sharing>). Экспортировать в pdf и открыть Pull Request в этот репозиторий.

Ход работы

Прохождение игры Flexbox Froggy

Во время прохождения игры мне удалось вспомнить то, как работают следующие синтаксические единицы CCS-a:

justify-content (выравнивание элементов по горизонтали)

align-items (выравнивание элементов по вертикали)

flex-direction (задает направление элементов в контейнере)

order (применяется для одного из элементов контейнера, позволяет указывать ему конкретный индекс размещения)

align-self (аналогичное действие, что и у align-items, но применяется только к одному элементу)

flex-wrap (отвечает за размещение элементов - их перенос на другую строку или наоборот намеренное сжатие)

flex-flow (объединение flex-wrap и flex-direction)

align-content (управление расположением рядами целого контейнера)

Ниже привела скрины с выполненными заданиями игры. Игру проходила во второй раз, первый был около 2 лет назад, к сожалению, знания синтаксиса пропали, пришлось изучать эти пункты заново.

Уровень 1 из 24

FLEXBOX FROGGY

Добро пожаловать в Flexbox Froggy! Игра, в которой тебе нужно помочь лягушечку Фроги и его друзьям, написав CSS код! Направь этого лягушонка на лисий остров, используя свойство **justify-content**, которое выравнивает элементы горизонтально и принимает следующие значения:

- flex-start**: элементы выравниваются по левой стороне контейнера.
- flex-end**: элементы выравниваются по правой стороне контейнера.
- center**: элементы выравниваются по центру контейнера.
- space-between**: элементы отображаются с одинаковыми отступами между ними.
- space-around**: элементы отображаются с одинаковыми отступами вокруг них.

Например, **justify-content: flex-end** сдвинет лягушонка вправо.

```
1 #pond {
2   display: flex;
3   justify-content: flex-end
4 }
5
6
7
8
9
10
```

Следующий

Игра Flexbox Froggy создана Codecademy • YouTube • Twitter • GitHub • Настройки

Хочешь изучить CSS grid? Курсinho в Grid Garden.



Уровень 2 из 24

FLEXBOX FROGGY

Используй **justify-content** ещё раз, чтобы помочь этим лягушкам попасть на их лиски. Помните, что это свойство CSS выравнивает элементы горизонтально и принимает следующие значения:

- flex-start**: элементы выравниваются по левой стороне контейнера.
- flex-end**: элементы выравниваются по правой стороне контейнера.
- center**: элементы выравниваются по центру контейнера.
- space-between**: элементы отображаются с одинаковыми отступами между ними.
- space-around**: элементы отображаются с одинаковыми отступами вокруг них.

```
1 #pond {
2   display: flex;
3   justify-content: center
4 }
5
6
7
8
9
10
```

Следующий

Игра Flexbox Froggy создана Codecademy • YouTube • Twitter • GitHub • Настройки

Хочешь изучить CSS grid? Курсinho в Grid Garden.



Уровень 3 из 24

FLEXBOX FROGGY

Помоги всем трём лягушкам найти их лиски, просто используя **justify-content**. В этот раз у лисий много пространства вокруг.

Если ты чувствуешь, что забыл возможные значения свойства, ты можешь навести курсор на название свойства, чтобы посмотреть их. Попробуй навести курсор на **justify-content**.

```
1 #pond {
2   display: flex;
3   justify-content: space-around
4 }
5
6
7
8
9
10
```

Следующий

Игра Flexbox Froggy создана Codecademy • YouTube • Twitter • GitHub • Настройки

Хочешь изучить CSS grid? Курсinho в Grid Garden.



Уровень 4 из 24

FLEXBOX FROGGY

Теперь лиски по краям упились к берегам, увеличив пространство между ними. Используй **justify-content**. В этот раз у лисий одинаковое расстояние между ними.

```
1 #pond {
2   display: flex;
3   justify-content: space-between
4 }
5
6
7
8
9
10
```

Следующий

Игра Flexbox Froggy создана Codecademy • YouTube • Twitter • GitHub • Настройки

Хочешь изучить CSS grid? Курсinho в Grid Garden.



Теперь, используя **align-items**, чтобы помочь лягушат добраться до нижней части пруда. Это свойство CSS выравнивает элементы вертикально и принимает следующие значения:

- **flex-start**: элементы выравниваются по верхнему краю контейнера.
- **flex-end**: элементы выравниваются по нижнему краю контейнера.
- **center**: элементы выравниваются вертикально по центру контейнера.
- **baseline**: элементы отображаются на базовой линии контейнера.
- **stretch**: элементы растягиваются, чтобы заполнить контейнер.

```
1 #pond {
2   display: flex;
3   align-items: flex-end
4 }
5
6
7
8
9
10
```

Игра Flexbox Froggy создана [Codrops](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Настройки](#)

Хотите изучить CSS grid? Курсите в [Grid Garden](#).



FLEXBOX FROGGY

Уровень 6 из 24

Направь лягушонка в центр пруда, используя **justify-content** и **align-items** вместе.

```
1 #pond {
2   display: flex;
3   justify-content: center;
4   align-items: center
5 }
6
7
8
9
10
```

Игра Flexbox Froggy создана [Codrops](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Настройки](#)

Хотите изучить CSS grid? Курсите в [Grid Garden](#).



FLEXBOX FROGGY

Уровень 7 из 24

Лягушатом снова нужно пересечь пруд. В этот раз к лискам, с достаточно большим пространством вокруг них. Используя комбинацию **justify-content** и **align-items**.

```
1 #pond {
2   display: flex;
3   justify-content: space-around;
4   align-items: flex-end
5 }
6
7
8
9
10
```

Игра Flexbox Froggy создана [Codrops](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Настройки](#)

Хотите изучить CSS grid? Курсите в [Grid Garden](#).



FLEXBOX FROGGY

Уровень 8 из 24

Лягушатом нужно выстроиться в том же порядке, что и лиски, используя **flex-direction**. Это свойство CSS задает направление, в котором будут расположены элементы в контейнере, и принимает следующие значения:

- **row**: элементы размещаются по направлению текста.
- **row-reverse**: элементы отображаются в обратном порядке к направлению текста.
- **column**: элементы располагаются сверху вниз.
- **column-reverse**: элементы располагаются снизу вверх.

```
1 #pond {
2   display: flex;
3   flex-direction: row-reverse
4 }
5
6
7
8
9
10
```



FLEXBOX FROGGY

Уровень 9 из 24

Помоги лягушатам расположиться на своих лилиях, используя **flex-direction**. Это свойство CSS задает направление, в котором располагаются элементы в контейнере, и принимает следующие значения:

- **row**: элементы размещаются по направлению текста.
- **row-reverse**: элементы отображаются в обратном порядке к направлению текста.
- **column**: элементы располагаются сверху вниз.
- **column-reverse**: элементы располагаются снизу вверх.

```
1 #pond {
2   display: flex;
3   flex-direction: column
4 }
5
6
7
8
9
10
```

Следующий



FLEXBOX FROGGY

Уровень 10 из 24

Помоги лягушатам попасть на свои лилии. Хотя и кажется, что они почти на своих местах, всё же придётся применить и **flex-direction**, и **justify-content**, чтобы поместить их на свои лилии.

Заметь, что когда ты устанавливаешь направление в обратном порядке для ряда или колонки, начало (start) и конец (end) тоже меняются местами.

```
1 #pond {
2   display: flex;
3   flex-direction: row-reverse;
4   justify-content: flex-end;
5 }
6
7
8
9
10
```

Следующий



FLEXBOX FROGGY

Уровень 11 из 24

Помоги лягушатам найти их лилии с помощью **flex-direction** и **justify-content**.

Заметь, когда в качестве направления выбрана колонка, то **justify-content** влияет на вертикальное выравнивание, а **align-items** — на горизонтальное.

```
1 #pond {
2   display: flex;
3   flex-direction: column;
4   justify-content: flex-end;
5 }
6
7
8
9
10
```

Следующий



Игра Flexbox Froggy создана [Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Настройки](#)

Хотите изучить CSS grid? Сыграйте в [Grid Garden](#).



Следующий



Следующий

Следующий

FLEXBOX FROGGY

Уровень 19 из 24

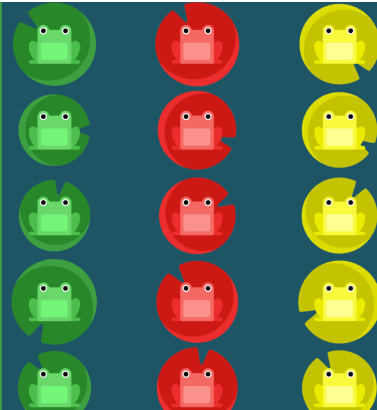
Помоги этой армии лягушек выстроиться в три колонки с помощью комбинации **flex-direction** и **flex-wrap**.

```
1 #pond {
2   display: flex;
3   flex-direction: column;
4   flex-wrap: wrap;
5 }
6
7
8
9
10
```

Следующий

[Игра Flexbox Froggy создана Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Настройка](#)

[Хотите изучить CSS grid? Курсовые в Gold Garden.](#)



FLEXBOX FROGGY

← Уровень 21 из 24 →

Лисушат раскидало по всему пруду, но лисичи сгруппированы в верхней части. Ты можешь использовать **`align-content`**, чтобы указать, как несколько рядов должны отделяться друг от друга. Данное свойство принимает следующие значения:

- **`flex-start`**: ряды группируются в верхней части контейнера.
- **`flex-end`**: ряды группируются в нижней части контейнера.
- **`center`**: ряды группируются вертикально по центру контейнера.
- **`space-between`**: ряды отображаются с одинаковыми расстояниями между ними.
- **`space-around`**: ряды отображаются с одинаковыми расстояниями вокруг них.
- **`stretch`**: ряды растягиваются, чтобы заполнить контейнер равномерно.

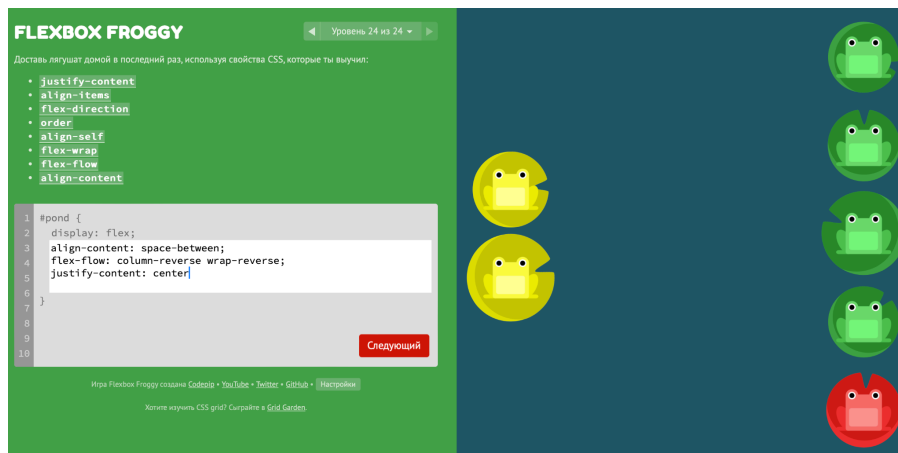
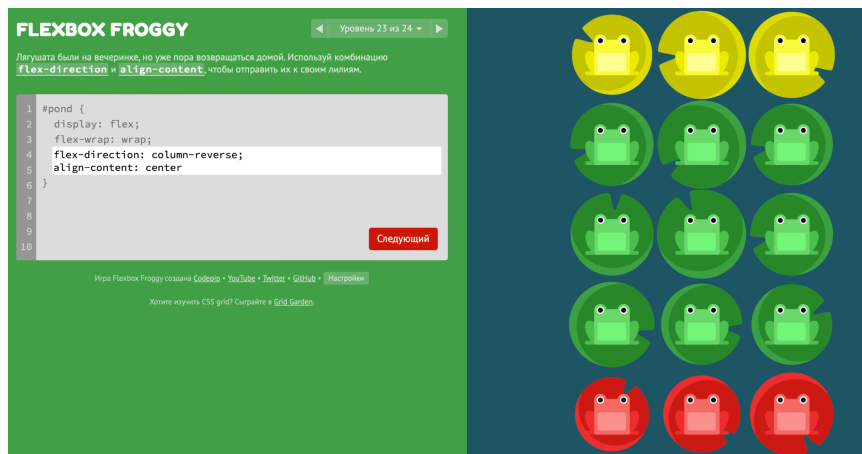
Это может запутать, но **`align-items`** отвечает за расстояние между рядами, в то время как **`align-self`** отвечает за то, как элементы в целом будут выровнены в контейнере. Когда только один ряд, **`align-content`** ни на что не влияет.

```

1 #pond {
2   display: flex;
3   flex-wrap: wrap;
4   align-content: flex-start;
5 }
6
7
8
9
10

```

[Следующий](#)



Прохождение игры Grid Garden

Аналогично, что и с игрой Flexbox Frog, по прохождении игры Grid Garden получили навыки работы с синтаксисом языка. В случае с этой игрой мы, узнали что такое grid, и научились с ним работать. Grid - это набор горизонтальных и вертикальных линий, которые создают шаблон для выстраивания дизайна. Основные единицы, с которыми мы познакомились:

- grid-column-start (определение начальной позиции столбца)
- grid-column-end (определение конечной позиции столбца)
- grid-row-start (определение начальной позиции ряда)
- grid-row-end (определение конечной позиции ряда)
- order (работа с конкретным рядом или столбцом)
- grid-template (определение колонок, рядов и области грид-раскладки)
- span (протяженность того или иного объекта)

Из нового и непонятного - работа с индексацией. Почему-то в каких-то кейсах индекс -1 вовсе не отображался на UI, где-то включительно, где-то не включительно берется последний и первый элементы. Также ниже прикладываю скрины с решением задач.

GRID GARDEN

Уровень 3 из 28

Когда вы используете только `grid-column-start`, `grid`-элемент по умолчанию "захватит" только один столбец. Однако вы можете увеличить элемент между несколькими столбцами, если добавите свойство `grid-column-end`.

Используя `grid-column-end`, полейте всю морковь, избегая пустые участки. Ведь мы не хотим использовать воду напрасно! Помните, что морковь начинается на первой вертикальной `grid`-линии и заканчивается на четвёртой.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column-start: 1;
9   grid-column-end: 4;
10 }
11
12
13
14
```

Следующий

GRID GARDEN

Уровень 4 из 28

Когда вы соединили `grid-column-start` и `grid-column-end`, вы могли подумать, что конечное значение обязательно должно быть больше начального. Но это не так!

Попробуйте задать свойству `grid-column-end` значение меньше, чем 5, чтобы полить морковь.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column-start: 5;
9   grid-column-end: -5;
10 }
11
12
13
14
```

Следующий

Мир Grid Garden создан [Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Русский](#)

Want to learn CSS Flexbox? Play [Flexbox Froggy](#).

GRID GARDEN

Уровень 5 из 28

Если вы хотите полить `grid`-строку справа налево вместо слева направо, вы можете задать `grid-column-start` и `grid-column-end` отрицательные значения. Например, вы можете присвоить значению `rowspan` `-3`, чтобы указать первую `grid`-строку начиная справа.

Попробуйте присвоить `grid-column-end` отрицательное значение.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column-start: 1;
9   grid-column-end: -2;
10 }
11
12
13
14
```

Следующий

Мир Grid Garden создан [Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Русский](#)

Want to learn CSS Flexbox? Play [Flexbox Froggy](#).

GRID GARDEN

Уровень 6 из 28

Теперь попробуйте присвоить `grid-column-start` отрицательное значение.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20%;
5 }
6
7 #poison {
8   grid-column-start: -3;
9 }
10
11
12
13
14
```

Следующий

Мир Grid Garden создан [Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Русский](#)

Want to learn CSS Flexbox? Play [Flexbox Froggy](#).

GRID GARDEN

Уровень 7 из 28

Вместо определения grid-элемента на основе начальной и конечной позиции grid-строк вы можете задавать их с помощью необходимой вам ширины строк, используя **span**. Помните, что **span** работает только с положительными значениями.

Для примера, полейте эту морковь, используя свойство **grid-column-end: span 2**.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column-start: 2;
9   grid-column-end: span 2
10 }
11
12
13
14
```

Следующий



GRID GARDEN

Уровень 8 из 28

Попробуйте опять использовать **grid-column-end** вместе со **span**, чтобы полить морковь.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column-start: 1;
9   grid-column-end: span 5
10 }
11
12
13
14
```

Следующий



GRID GARDEN

Уровень 9 из 28

Вы также можете использовать **span** вместе с **grid-column-start**, чтобы присвоить значение ширины grid-элемента относительно конечной позиции.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column-start: span 3
9   grid-column-end: 6;
10 }
11
12
13
14
```

Следующий



GRID GARDEN

Уровень 10 из 28

Печать каждый раз **grid-column-start + grid-column-end** может быть немного утомительно. К счастью, есть краткая форма **grid-column**, которая принимает оба значения сразу через косую черту: **/**.

Например, **grid-column: 2 / 4** скажет grid-элементу начаться на второй вертикальной grid-линии и закончиться на четвёртой.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column: 4 / 6;
9 }
10
11
12
13
14
```

Следующий



GRID GARDEN

Уровень 11 из 28

Попробуйте использовать `grid-column`, чтобы полить эту морковь. `span` также работает с этим сокращённым свойством, так что попробуйте!

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column: 2 / span 3;
9 }
10
11
12
13
14
```

Следующий



GRID GARDEN

Уровень 12 из 28

Одна из вещей, которая отличает CSS Grid Layout от Flexbox — это то, что вы можете легко позиционировать элементы в двух направлениях: в столбцах и в строках. `grid-row-start` работает почти как `grid-column-start`, но только по вертикальной оси.

Используйте `grid-row-start`, чтобы полить эту морковь.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-row-start: 3;
9 }
10
11
12
13
14
```

Следующий



GRID GARDEN

Уровень 13 из 28

А теперь попробуйте использовать сокращённое свойство `grid-row`.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-row: 3 / 6;
9 }
10
11
12
13
14
```

Следующий



Игра Grid Garden создана [Codepen](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Русский](#)

Want to learn CSS flexbox? Play [Flexbox Froggy](#).

GRID GARDEN

Уровень 14 из 28

Используйте `grid-column` и `grid-row` одновременно, чтобы разместить элемент в двух направлениях.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #poison {
8   grid-row: 5;
9   grid-column: 2;
10 }
11
12
13
14
```

Следующий

Игра Grid Garden создана [Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Русский](#)

Want to learn CSS flexbox? Play [Flexbox Froggy](#).

GRID GARDEN

Уровень 15 из 28

Вы также можете использовать `grid-column` и `grid-row` вместе, чтобы охватить более крупные зоны внутри grid-сетки. Попробуйте!

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column: 2 / span 6;
9   grid-row: 1 / span 6;
10 }
11
12
13
14
```

Следующий

Игра Grid Garden создана [Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Русский](#)

Want to learn CSS flexbox? Play [Flexbox Froggy](#).

GRID GARDEN

Уровень 16 из 28

Если вы также хотите указать конкретные `grid-column` и `grid-row`, есть сокращённое свойство и для этого `grid-area` принимает 4 значения, разделённые косой чертой `/`: `grid-row-start` `grid-column-start` `grid-row-end` `grid-column-end`.

Пример: `grid-area: 1 / 1 / 3 / 6`;

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-area: 1 / 2 / 4 / 6;
9 }
10
11
12
13
14
```

Следующий

Игра Grid Garden создана [Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Русский](#)

Want to learn CSS flexbox? Play [Flexbox Froggy](#).

GRID GARDEN

Уровень 17 из 28

Как насчет множества элементов? Вы можете накладывать их друг на друга без проблем. Используйте `grid-area`, чтобы определить вторую зону, которая покрывает всю не политую морковь.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water-1 {
8   grid-area: 1 / 4 / 6 / 5;
9 }
10
11 #water-2 {
12   grid-area: 2 / 3 / 5 / 4;
13 }
14
```

Следующий

Игра Grid Garden создана [Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Русский](#)

Want to learn CSS Flexbox? Play [Flexbox Froggy](#).



GRID GARDEN

Уровень 18 из 28

Если grid-элементы не имеют конкретного расположения с `grid-area`, `grid-column`, `grid-row` и т.д., они автоматически размещаются, следуя порядку, написанному в коде. Мы можем изменить это с помощью свойства `order`, которое является одним из преимуществ CSS Grid Layout перед табличной разметкой.

Изначально все grid-элементы имеют `order`, равный 0, но этому свойству можно присвоить любое положительное или отрицательное значение, примерно как у `z-index`.

Сейчас морковь во втором столбце оравлена и сорняки в последнем столбце поливаются. Измените значение свойства `order` для яды, чтобы исправить это прямо сейчас!

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 .water {
8   order: 0;
9 }
10
11 #poison {
12   order: 5;
13 }
14
```

Следующий

Игра Grid Garden создана [Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Русский](#)



GRID GARDEN

Уровень 19 из 28

Теперь вода и яд меняются местами, так как все сорняки находятся в начале сада. Присвойте правильное значение `order` для яды, чтобы исправить это.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 .water {
8   order: 0;
9 }
10
11 .poison {
12   order: -1;
13 }
14
```

Следующий



GRID GARDEN

Уровень 20 из 28

До этого момента сад имел grid-сетку с пятью столбцами по 20% ширины и пятью строками, каждая по 20% высоты.

Это было сделано свойствами `grid-template-columns: 20% 20% 20% 20% 20%;` и `grid-template-rows: 20% 20% 20% 20% 20%;`. Каждое свойство имеет пять значений, которые создают пять столбцов, где ширина каждого равна 20% от общей ширины сада.

Но вы можете изменить grid-сетку как вам вздумается. Присвойте `grid-template-columns` новое значение, чтобы полить морковь. Вам необходимо поставить значение ширины первого столбца равным 50%.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 50% 20% 20% 20% 20%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column: 1;
9   grid-row: 1;
10 }
11
12
13
14
```

Следующий

Игра Grid Garden создана [Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Русский](#)

Want to learn CSS Flexbox? Play [Flexbox Froggy](#).



GRID GARDEN

Уровень 21 из 28

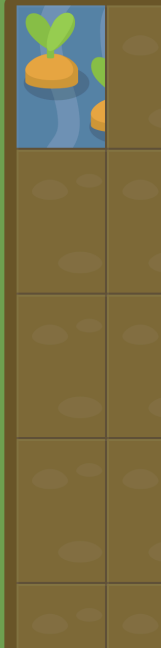
Определять несколько столбцов с одинаковой шириной может быть утомительно. К счастью, есть функция **repeat**, которая призвана помочь в этом.

Например, раньше мы определяли пять столбцов по 20% ширины с помощью **grid-template-columns: 20% 20% 20% 20% 20%;** Это можно упростить до **grid-template-columns: repeat(5, 20%);**

Используя **grid-template-columns** вместе с функцией **repeat**, создайте 8 столбцов по 12.5% ширины каждый. Таким образом, вы не будете использовать больше воды, чем нужно.

```
1 #garden {
2   display: grid;
3   grid-template-columns: repeat(8, 12.5%)
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-column: 1;
9   grid-row: 1;
10 }
11
12
13
14
```

Следующий



GRID GARDEN

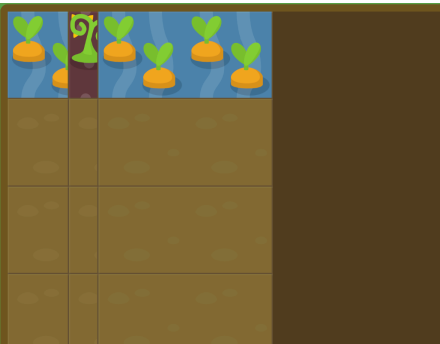
Уровень 22 из 28

grid-template-columns принимает значения не только в процентах, но и в единицах длины, например **px** или **em**. Вы даже можете комбинировать разные единицы измерения.

Присвойте трем столбцам значение **100px**, **3em** и **40%** соответственно.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 100px 3em 40%;
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7
8
9
10
11
12
13
14
```

Следующий



GRID GARDEN

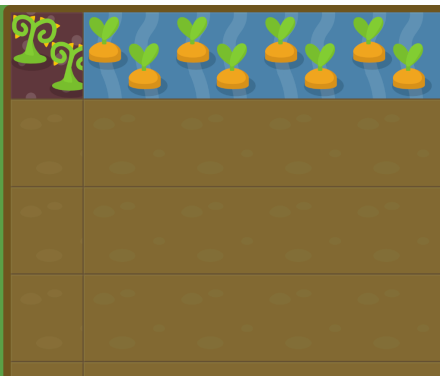
Уровень 23 из 28

CSS Grid Layout также вводит новую единицу измерения: дробный **fr** (англ. *fraction*). Каждый **fr** выделяет одну часть свободного пространства. Например, если двум элементам задано **1fr** и **3fr**, соответственно, то пространство будет поделено на 4 одинаковые части. Первый элемент займет 1/4, а второй оставшиеся 3/4 пространства.

Сейчас сорняки занимают левую 1/4 часть вашей первой строки, а морковь оставшиеся 3/4. Создайте два столбца с такой же шириной, используя **fr**.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 1fr 5fr
4   grid-template-rows: 20% 20% 20% 20% 20%;
5 }
6
7
8
9
10
11
12
13
14
```

Следующий



GRID GARDEN

Уровень 24 из 28

Когда одни столбцы определены с помощью пикселей, процентов или `em`, а любые другие столбцы – с помощью `fr`, то последние просто разделят всё возможное оставшееся пространство.

Сейчас морковь занимает 50 пикселей слева, а сорняки 50 пикселей справа. С помощью **grid-template-columns** создайте два столбца и используйте `fr`, чтобы сделать ещё 3 столбца, которые займут оставшееся пространство между ними.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 50px repeat(3, 1fr) 50px;
4   grid-template-rows: 20% 20% 20% 20%;
5 }
6
7 #water {
8   grid-area: 1 / 1 / 6 / 2;
9 }
10
11 #poison {
12   grid-area: 1 / 5 / 6 / 6;
13 }
14
```

Следующий

Игра Grid Garden создана [Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Русский](#)

Want to learn CSS flexbox? Play [Flexbox Froggy](#).



GRID GARDEN

Уровень 25 из 28

Теперь у нас есть столбец сорняков с шириной 75 пикселей на левой стороне сада. 3/4 оставшегося пространства занимает растущая морковь, а 1/4 заполнили сорняки.

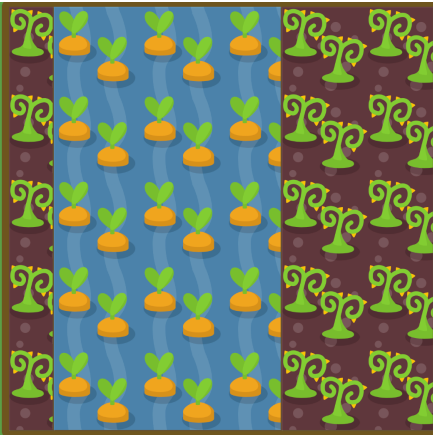
Используйте **grid-template-columns** с комбинацией `px` и `fr`, чтобы создать необходимые столбцы.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 75px 3fr 2fr;
4   grid-template-rows: 100%;
5 }
6
7
8
9
10
11
12
13
14
```

Следующий

Игра Grid Garden создана [Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Русский](#)

Want to learn CSS flexbox? Play [Flexbox Froggy](#).



GRID GARDEN

Уровень 26 из 28

grid-template-rows работает точно так же, как и **grid-template-columns**.

Используйте **grid-template-rows**, чтобы полить всё, кроме верхних 50 пикселей сада. Помните, что вода заполняет только пятую строку, так что вам нужно создать именно пять строк.

```
1 #garden {
2   display: grid;
3   grid-template-columns: 20% 20% 20% 20% 20%;
4   grid-template-rows: 50px 0 0 0 1fr;
5 }
6
7 #water {
8   grid-column: 1 / 6;
9   grid-row: 5 / 6;
10 }
11
12
13
14
```

Следующий

Игра Grid Garden создана [Codecademy](#) • [YouTube](#) • [Twitter](#) • [GitHub](#) • [Русский](#)

Want to learn CSS flexbox? Play [Flexbox Froggy](#).





Прохождение игры Git

Несмотря на уже более 2 лет опыта работы с гитом, большей части того функционала, что продемонстрировали в игре, я не использовала или использовала крайне редко. С такими функциями гита удалось ознакомиться или вновь повторить:

git commit (Фиксирует изменения в локальном репозитории, сохраняя текущее состояние индексированных файлов)

git branch (Управляет ветками. Позволяет создавать, удалять или отображать ветки в репозитории)

git checkout (Переключается между ветками или восстанавливает файлы из определенного коммита)

git merge (Объединяет изменения из одной ветки в другую, создавая новый коммит слияния)

git rebase (Переносит изменения с одной ветки на другую, изменяя историю коммитов и избегая слияния с дополнительным коммитом)

git reset (Откатывает указатель текущей ветки к предыдущему состоянию.

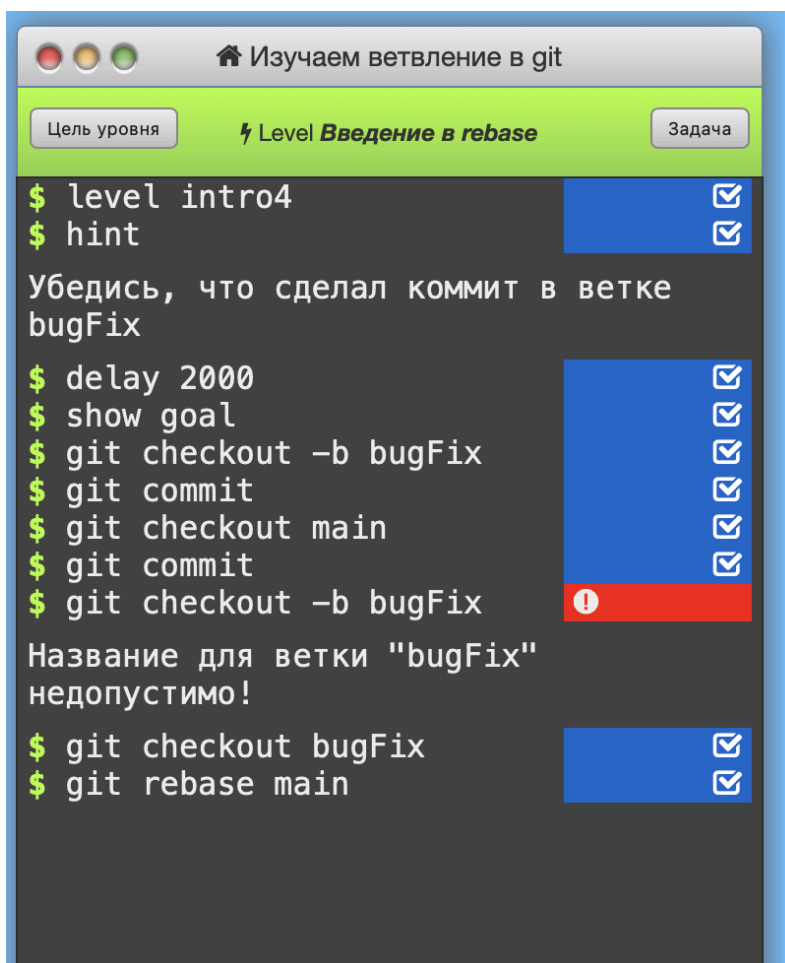
Может изменить также и рабочие файлы, и индекс)

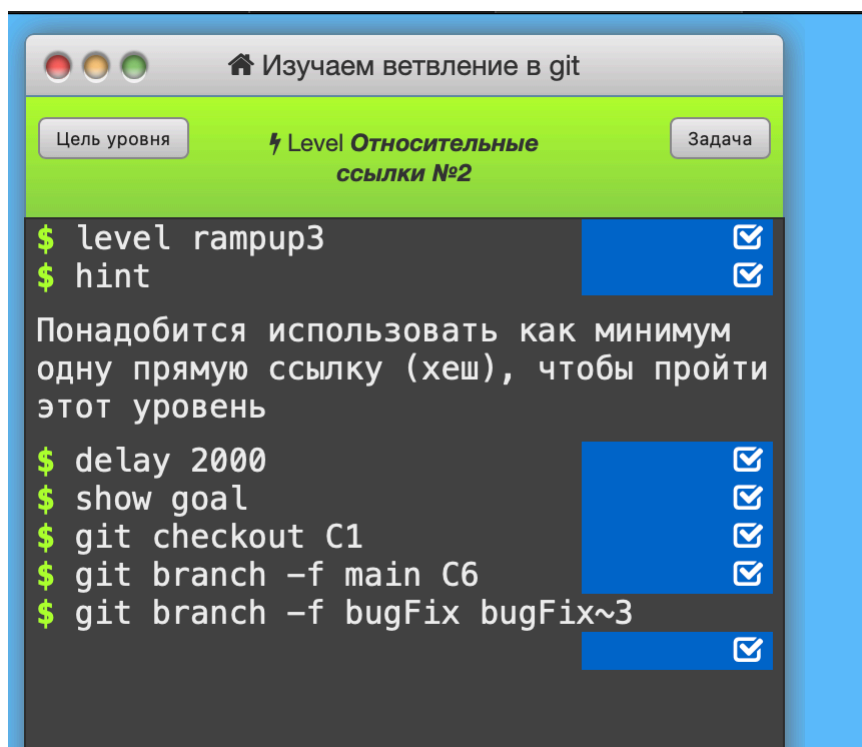
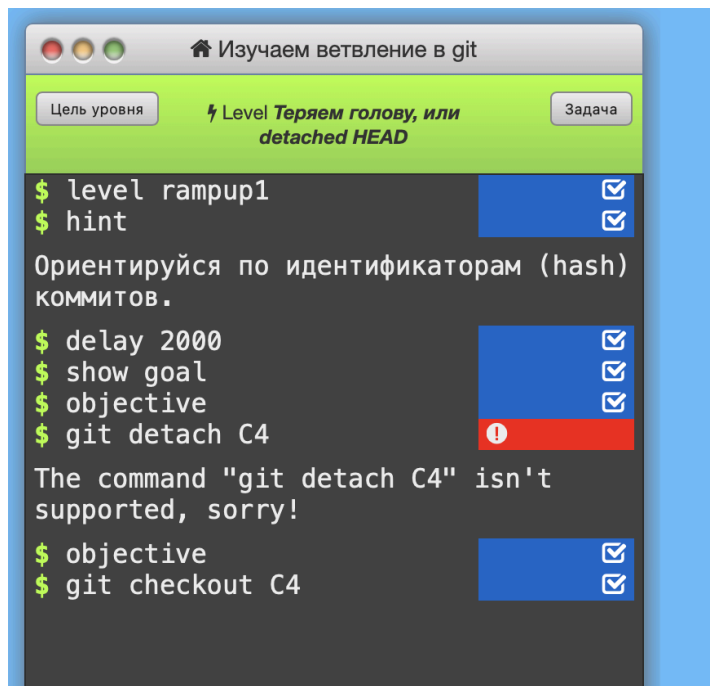
git revert (Создает новый коммит, который отменяет изменения из выбранного коммита, сохраняя при этом историю)

git cherry-pick (Применяет отдельные коммиты из одной ветки в другую, выборочно перенося изменения)

git tag (Создает метку (тег) на определенном коммите, обычно используется для версионирования)

Привожу скрины выполнения заданий ниже.





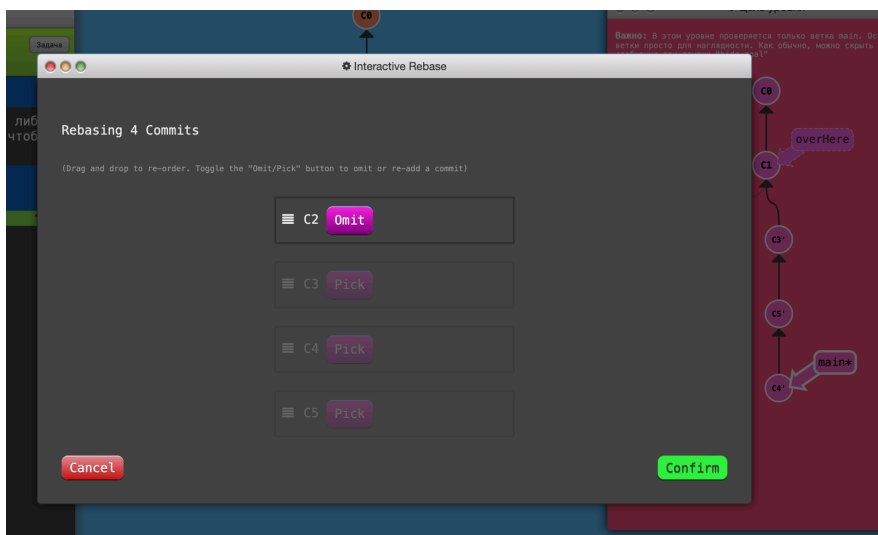
принимают разные параметры.

```
$ delay 2000
$ show goal
$ git reset HEAD
$ git reset HEAD~1
$
$ git checkout pushed
$ git revert pushed
```



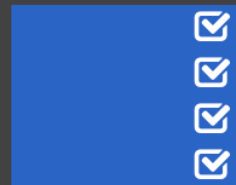
КОММИТОВ !

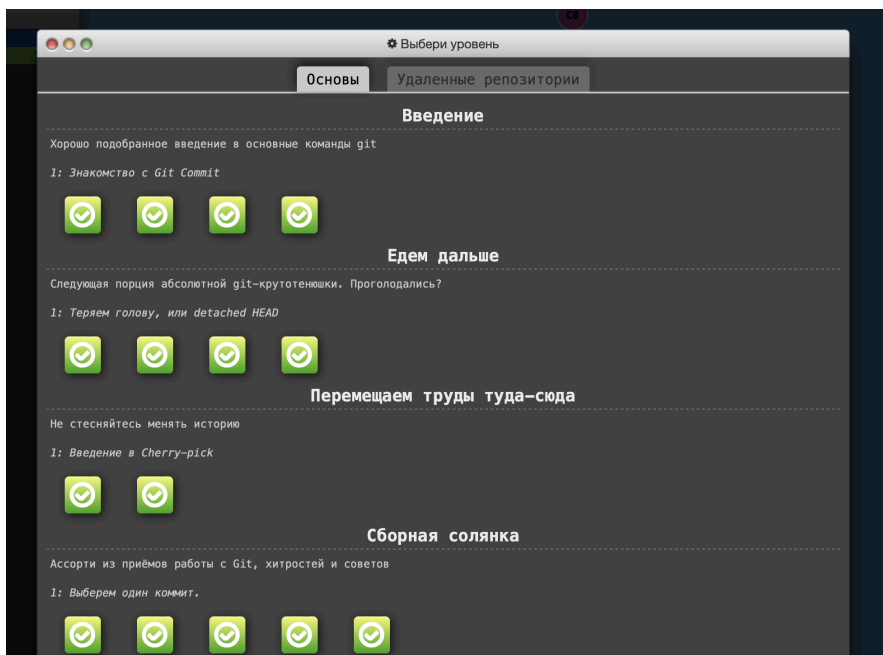
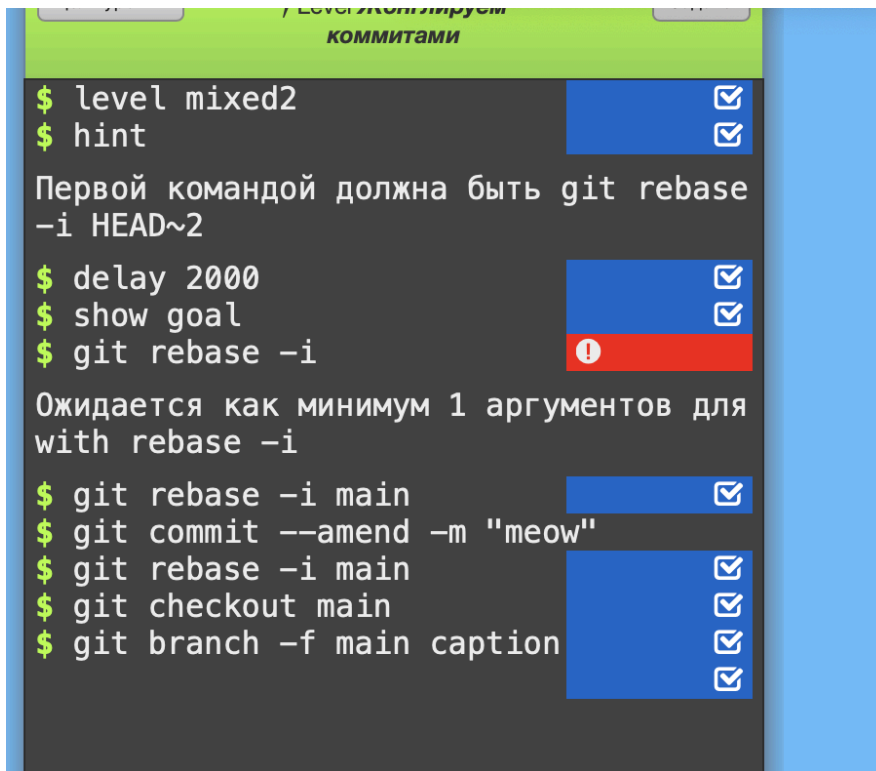
```
$ delay 2000
$ show goal
$ git cherry-pick C3 C4 C7
```

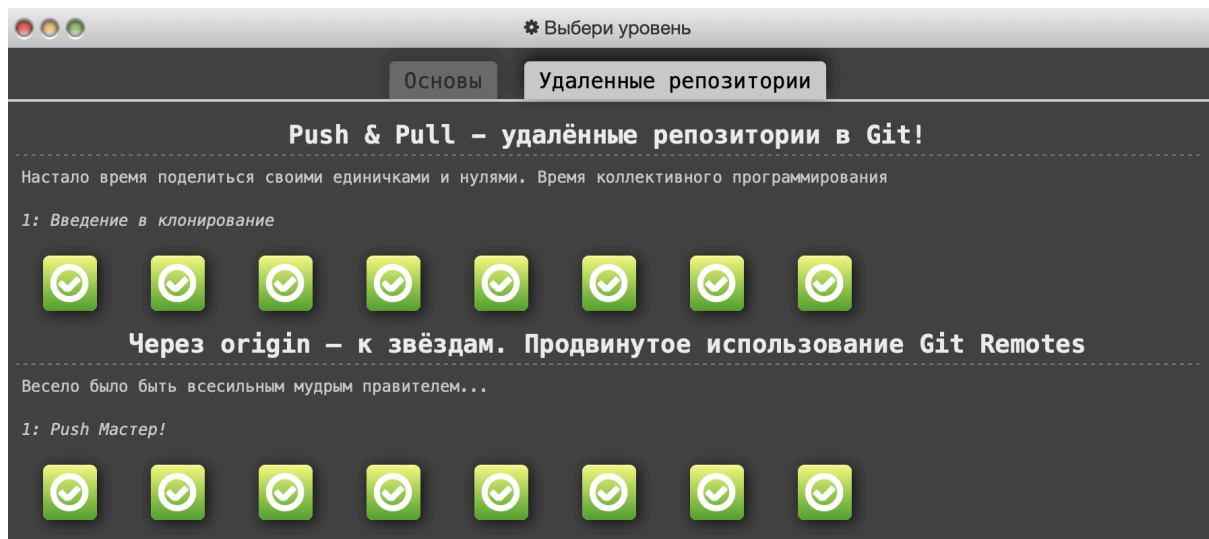


cherry-pick – это твои друзья!

```
$ delay 2000
$ show goal
$ git checkout main
$ git cherry-pick C4
```







Вывод

Пройдя эти три игры, я вспомнила синтаксис CSS, узнала о новом его функционале, а также вспомнила команды git, которыми не пользуюсь на регулярной основе.